Intro to ML Systems & Data Engineering

Overview

- explain the gap between machine learning (ML) models and products
- contrast engineering and data over just algorithms
- identify how to leverage cloud technologies for hands-on practice
- recognize the specifications and requirement on real-world enterprise ML scenarios

Course Requirements

- python 3.9+ running in a virtual environment
- git + github
- VS Code, PyCharm, or equivalent IDE
- jupyter notebook
- azure account

Readings

Sato, D. (n.d.). *Continuous delivery for machine learning*. martinfowler.com. https://martinfowler.com/articles/cd4ml.html

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J., & Dennison, D. (2015). Hidden technical debt in machine learning systems. **https://papers.nips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf**

Async	
Intro to ML Systems and Data Engineering	

Prototype vs Production	**Prototype.**
	• answers “can this approach work?”
	Production.
	• must continuously
	  ○ process data
	  ○ save predictions
	  ○ handle failures
	  ○ scale with demand
	  ○ protect secrets
	  ○ remain maintainable

ML Lifecycle	**These stages form a cycle rather than a one-time workflow.**
	• problem farming
	  ○ define the prediction or decision problem, business objective, success metrics, and operational constraint
	• data collection and preparation
	  ○ acquire, clean, transform, validate, and continuously process data
	• model development
	  ○ select algorithms, train models, tune parameters, and evaluate performance
	• deployment
	  ○ make predictions available through APIs, applications, cloud services, or other systems
	• monitoring and maintenance

	○ track model and system performance, detect changes in data, respond to failures, and retrain when necessary ○ monitoring may reveal new problems that require revisiting the data, model, or even the original problem definition
ML Systems vs. Traditional Software	**Machine learning introduces challenges that are less prominent in traditional software development.** • model outputs are often probabilistic rather than deterministic • system behavior can change when the underlying data changes • production systems must manage multiple evolving artifacts—including: ○ code ○ datasets ○ features ○ trained models • testing therefore includes not only software correctness, but also: ○ statistical model performance ○ data validation ○ latency ○ throughput ○ bias ○ other operational measures

Core Components of a Production ML System	**A typical ML system contains several interconnected components.**
	• data ingestion pipelines
	  ○ batch
	  ○ streaming
	  ○ on-demand data
	• feature engineering and storage for transforming raw data into usable model inputs
	• training pipelines that make model training repeatable and reproducible
	• model serving infrastructure
	  ○ APIs
	  ○ containers
	  ○ managed cloud endpoints
	• monitoring and feedback loops
	  ○ detect problems
	  ○ collect new information for future training
	
	Together, these components turn a trained model into a continuously operating system.

Why Data Engineering Matters	**Production ML depends heavily on the quality and availability of data.**
	• poor, incomplete, biased, or corrupted data can undermine even an excellent model.
	
	• data engineering therefore focuses on:
	  ○ reliable ingestion
	  ○ validation

	○ storage ○ transformation ○ scheduling ○ integration across potentially very large datasets • important data-related risks include: ○ distribution shift ○ data poisoning ○ poor label quality ○ privacy / compliance constraints **Designing ML systems requires anticipating these failure modes instead of treating data as a fixed input.**
The Role of Cloud Infrastructure	• cloud platforms make many production ML workloads easier to operate by providing: ○ elastic computing resources ○ managed services ○ geographic distribution ○ autoscaling ○ integrated data ecosystems • these capabilities allow teams accomplish within a common infrastructure: ○ training large models ○ serving predictions at scale ○ connect: ▪ storage ▪ analytics ▪ ML workflows

Key Takeaway	**Production machine learning is a systems engineering problem, not just a modeling problem.**
	• model accuracy remains important, but a useful ML product must also be:
	  ○ reliable
	  ○ scalable
	  ○ maintainable
	  ○ secure
	  ○ supported by high-quality data

ML Success Hinges on Hidden Infrastructure	

Engineering Mindset and Real-World ML Systems	• useful ML system must be designed not only around model quality, but also around:
	  ○ reliability
	  ○ security
	  ○ maintainability
	  ○ data quality
	  ○ infrastructure
	  ○ integration with the surrounding application
	• in practice:
	  ○ the model itself may represent only a small portion of the overall system
	• much of the work lies in:
	  ○ supporting pipelines
	  ○ features

	○ testing ○ infrastructure
An Engineering Mindset	• this course encourages thinking beyond experimentation and asking questions such as: ○ will this work reliably in production? ○ how will it be maintained? ○ is it secure? ○ what happens when the data or operating environment changes? ○ how will users or other systems interact with the model? **The goal is to connect ML theory with practical implementation using modern cloud systems.**
Real-World Systems Have Different Requirements	• system requirements depend on the application: ○ recommendation systems combine: ▪ behavioral data pipelines ▪ low-latency serving ▪ feedback loops that adapt as user preferences change ○ fraud detection requires: ▪ real-time streaming data ▪ rapidly computed features ▪ very low prediction latency ▪ extremely high availability ○ predictive maintenance often relies on:

        - large-scale sensor data
        - batch processing
        - integration with operational workflows such as maintenance scheduling

    - computer vision systems may require:
        - large-scale storage
        - GPU inference
        - high throughput
        - in sensitive domains such as medicine—strong requirements for:
            - accuracy
            - explainability
            - security
            - compliance

- there is no single definition of a “good” ML system
- different applications prioritize different qualities
    - latency
    - scale
    - availability
    - interpretability
    - regulatory compliance

The Lifecycle as an Iterative Process	• building an ML system begins with problem framing
	  ○ defining the decision or prediction task
	  ○ establishing meaningful measures of success
	  ○ identifying constraints before selecting a model
	
	• data must then be:
	  ○ collected
	  ○ cleaned
	  ○ transformed
	  ○ converted into useful features
	• in production, these transformations should become repeatable pipelines so that new data is processed consistently rather than prepared manually each time
	
	• model development
	  ○ remains important
	  ○ but it is only one stage of the larger workflow.
	• **o**nce a model is considered suitable it must be**:**
	  ○ deployed
	  ○ integrated into a service or application
	  ○ scaled appropriately
	  ○ continuously monitored and maintained
	
	• production systems also require a plan for changing conditions
	  ○ new user behavior
	  ○ shifts in the environment
	  ○ declining model performance
	  ○ harmful predictions
	  ○ which may require:

	▪ retraining ▪ new data ▪ rollback procedures ▪ possibly reframing the original problem • for this reason, the ML lifecycle is best understood as: ○ a continuous feedback loop ○ NOT a linear sequence with a final endpoint
Machine Learning Systems	
Testing ML Systems	**quality assurance for machine learning is broader than conventional software testing** • in addition to functional correctness, ML systems must be evaluated for: ○ system performance: ▪ latency ▪ throughput ▪ resource usage ▪ availability ○ model performance: ▪ accuracy ▪ false-positive rates ▪ other task-specific metrics ○ data quality: ▪ malformed ▪ missing

	▪ unexpected ▪ or changing inputs ○ bias and edge cases: ▪ behavior that may not appear in ordinary functional tests • because both data and models can evolve after deployment, production testing may also involve techniques such as data validation and shadow deployments for evaluating new models before fully replacing the existing one
Functional Components of an ML System	• a production system can be decomposed into several connected layers: **Data ingestion.** ▪ brings raw information into the system ▪ depending on the application, ingestion may be: • streaming • batch-based • triggered on demand ▪ its responsibility is to ensure that the system continuously receives the data needed for downstream processing **Feature engineering and storage.** ▪ convert raw data into model-ready representations production systems ▪ may precompute and store frequently used features,

- sometimes through a dedicated feature store so that identical feature definitions can be used during both training and inference

Training pipelines.

- turn model development into an automated, repeatable process
- they can include:
  - dataset splitting
  - training
  - hyperparameter tuning
  - validation
  - eventually, continuous retraining as new data becomes available

Model serving.

- exposes the trained model to other applications
  - commonly through an API
- serving infrastructure must be selected according to operational requirements such as:
  - latency
  - scale
  - availability

Monitoring and feedback loops.

- connect production behavior back to the development process
- monitoring

	• tracks both infrastructure health and changes in model or data behavior ▪ feedback • supplies eventual outcomes or user responses that can support evaluation and retraining
Data Engineering as the Foundation	**A model is only as dependable as the data pipeline supporting it.** • production data engineering therefore includes ○ validation rules ○ missing-data handling ○ anomaly detection ○ workflow scheduling ○ mechanisms for enforcing the quality requirements expected by ML models • the model must also operate at realistic scale ○ enterprise systems may process millions of events or hundreds of millions of records requiring ▪ distributed processing frameworks and cloud infrastructure ▪ not achievable with single-machine workflows
Data Lineage and Governance	**An additional production concern is understanding where data came from and how it is allowed to be used.** • data lineage ○ records the origin and transformation history of data, making it easier to trace downstream problems back to their source

	• data governance
	  ○ defines and enforces:
	  ▪ rules around access
	  ▪ privacy
	  ▪ compliance
	  ▪ anonymization
	  ▪ appropriate use
	• these controls become particularly important when ML systems use sensitive or regulated information

Data-Specific Failure Modes	• *several problems can undermine model performance even when the modeling code itself is unchanged*
	  • ***data poisoning***
	  ○ *occurs when harmful or misleading observations enter the training or inference pipeline, potentially intentionally*
	  ○ *systems that learn from open or user-generated data are particularly exposed to this class of risk*
	  • ***poor label quality***
	  ○ *supervised models trained on incorrect or inconsistent labels are effectively learning from noise*
	  ○ *label errors*
	  ▪ *reduce attainable performance*
	  ▪ *introduce unwanted bias*
	These issues reinforce the need to treat the training dataset itself as an engineered and monitored component of the system.

Cloud Infrastructure	• benefits of cloud infrastructure:
	  ○ elastic compute for scaling resources on demand
	  ○ managed ML services that reduce infrastructure overhead
	  ○ global reach through geographically distributed infrastructure
	  ○ unified data ecosystems that connect
	  ▪ storage
	  ▪ warehouses
	  ▪ lakehouse architectures
	  ▪ analytics
	  ▪ ML tools

Key Takeaways	**Production ML can be understood as a set of *interdependent engineering components*.** • reliability depends on how well data ingestion, feature computation, training, serving, monitoring, governance, and infrastructure work together—not on the model in isolation
Requirements for Production ML	

Static Data vs. Dynamic Data	• in research
	  ○ models are commonly developed on a fixed dataset with known boundaries
	  ○ after the train/test split is created, the data environment is relatively stable
	• production systems operate differently
	  ○ mew observations continue to arrive
	  ○ user behavior changes
	  ○ seasonality appears

	○ the underlying data distribution may drift ▪ as a result, production ML requires plans for: • periodic or continuous retraining • robust data ingestion pipelines • validation and cleaning of incoming data • handling missing or unavailable data sources • graceful degradation or fallback behavior when dependencies fail ○ the key distinction ▪ research • often treats data as a snapshot ▪ production • treats data as an ongoing stream that can change over time
Model Metrics Are Not Enough	• research workflows often judge models primarily using predictive metrics such as: ○ accuracy ○ F1 score ○ RMSE ○ other task-specific evaluation measures • production systems ○ still care about these metrics ○ they must balance them against operational requirements • important production criteria include: ○ latency

	▪ how quickly predictions are returned ○ throughput ▪ how many requests the system can process ○ scalability ▪ whether the system can handle increasing demand ○ resource usage and cost ○ uptime and reliability • a model with excellent offline accuracy may still be unsuitable if it is ○ too slow ○ too expensive ○ or unreliable in deployment
Continuous Testing and Quality Assurance	**Research testing is often centered on evaluation against a holdout dataset and a small number of sanity checks.** • Production ML requires a broader and continuous testing process ○ pre-production testing ○ canary or shadow deployments ○ continuous monitoring and alerting ○ security testing ○ ongoing checks of data and model behavior • testing therefore continues after deployment rather than ending once a model passes offline evaluation
Key Takeaways	• a production ML system must balance predictive quality with: ○ changing data ○ operational constraints

	○ continuous validation • the important shift is from optimizing a model once on static data to operating a system that remains: ○ accurate ○ responsive ○ reliable ○ testable as conditions change
ML Prototype, ML Product and ML System	
Machine Learning Prototypes	**A machine learning prototype is an early-stage demonstration of an idea. It is often a proof-of-concept model built by a data scientist.** • a prototype typically: ○ runs on a laptop or a single virtual machine ○ uses a clean sample of the available data ○ tests whether an ML approach is feasible ○ is evaluated internally rather than used by customers ○ prioritizes rapid experimentation over reliability ○ may require manual adjustments ○ may have limited testing, security, and code organization • occasional crashes or manual intervention are usually acceptable because the prototype's purpose is to answer one question
Machine Learning Products	**An ML product incorporates a promising model into a live service, system, or application feature.** • the model is no longer isolated in a notebook. It becomes one component within a larger production pipeline

	• a production ML product may need to:
	  ○ receive data continuously
	  ○ generate predictions for users or automated processes
	  ○ operate reliably at all times
	  ○ serve predictions through an application or API
	  ○ handle increasing numbers of users and requests
	
	• the central question changes from:
	  ○ can the model work?
	• to:
	  ○ can the complete system serve users reliably, securely, and at scale?
	
	• a successful prototype may represent only about 10% of the journey. The remaining work involves converting the prototype into a dependable production system

Production Engineering Requirements	**Model management.**
	• teams need procedures and tools to:
	  ○ deploy updated models
	  ○ maintain model versions
	  ○ track which model is currently active
	  ○ roll back to an earlier version when necessary
	  ○ reproduce previous training runs
	
	Monitoring.
	• a production system must continuously monitor:
	  ○ prediction quality

	○ model accuracy and other performance metrics ○ data quality ○ data-distribution changes ○ system errors and failures ○ latency and resource use • a model can continue operating normally from a technical perspective while its predictions gradually become less useful. **Security and privacy.** • production ML systems must protect both models and data • important controls include: ○ restricting access to the model ○ encrypting data in transit ○ encrypting stored data ○ protecting sensitive inputs and predictions ○ meeting relevant privacy and compliance requirements • for medical ML services, privacy, security, and regulatory compliance are essential rather than optional **Scalability.** • the system must handle increasing demand • for example, if an API receives ten times its normal traffic, the architecture should: ○ scale automatically, or ○ support the rapid addition of computing resources
ML Systems Versus	**Probabilistic rather than strictly deterministic.** • traditional software is usually deterministic:

Traditional Software	○ the same input produces the same output ○ a sorting algorithm should always return the same ordered list for the same input • machine learning models are different: ○ training randomness can produce slightly different models ○ outputs often represent probabilities or predictions ○ predictions are not guaranteed to be correct ○ errors can occur even when the software contains no code defect • this uncertainty must be treated as a normal property of the system
Testing Machine Learning Systems	• traditional software tests may assert: $$for\ input\ (x), the\ output\ must\ be\ (y)$$ • this is often inappropriate for ML models because valid predictions are statistical rather than exact **ML testing therefore evaluates performance across many examples.** • a test might require that: ○ accuracy remains above a defined threshold ○ false-positive and false-negative rates remain acceptable ○ performance satisfies requirements across important subgroups ○ confidence scores are appropriately calibrated ○ predictions remain stable under expected input variations **The surrounding system should also manage uncertain predictions.** • possible strategies include:

<table>
<tr><td></td><td>

applying decision thresholds
rejecting low-confidence predictions
sending uncertain cases for human review
using a fallback model or rule-based process

</td></tr>
<tr><td>Three Sources of Change</td><td>
Traditional software primarily changes when its code changes.

an ML system has three major sources of change:

<table>
<tr><th>Component</th><th>Example of change</th></tr>
<tr><td>Code</td><td>A developer updates the prediction service</td></tr>
<tr><td>Model</td><td>The model is retrained or replaced</td></tr>
<tr><td>Data</td><td>New user behavior, slang, hashtags, or market conditions appear</td></tr>
</table>
Model behavior can therefore change even when no code has been modified.

for example:

a model that identifies trending topics may become less accurate when new language or hashtags emerge
the incoming data distribution has changed

even though the application code remains identical

</td></tr>
<tr><td>Data Drift</td><td>
Data drift occurs when production data becomes different from the data used to train the model.

drift can:

change model behavior

</td></tr>
</table>

	○ reduce prediction quality ○ introduce new patterns that the model has never encountered ○ silently degrade performance without causing a software error **Because drift may not produce crashes or warning messages, it must be detected through continuous monitoring and evaluation.**
Continuous Training and Delivery	**MLOps, or machine learning operations, extends software-engineering practices to ML systems.** • a central MLOps principle is: ○ continuous training ○ continuous evaluation ○ continuous delivery • this resembles continuous integration and continuous delivery, or CI/CD, in traditional software engineering **MLOps must also account for changing data and retrained models.** • model updates should be carefully evaluated before deployment • a newly trained model is not automatically better simply because it uses newer data
Versioning and Reproducibility	**Traditional software projects usually version their source code with tools such as Git.** • ML systems must manage additional artifacts: ○ source code ○ training data

	○ validation and test data ○ data-processing procedures ○ model configurations and hyperparameters ○ training-run records ○ learned model parameters ○ evaluation results ○ deployed model versions • if a production model begins behaving unexpectedly, the team should be able to determine: ○ which code produced it ○ which dataset was used ○ how the data was processed ○ which parameters were selected ○ which training run created the model ○ which evaluation results justified its deployment **Without dataset and model versioning, reliable investigation and reproduction may be impossible.**
Ongoing Validation	**ML deployment is not the end of model development.** • a production model requires ongoing validation because: ○ real-world data changes ○ model performance can decline silently ○ user behavior changes ○ retraining can alter predictions ○ new security, fairness, or compliance risks may emerge

	As a result, the boundary between development and maintenance is less distinct in ML than in traditional software.
Brainstorming Time	

Model and Algorithm Selection	**Research models are often selected primarily for predictive performance.**
	• a complex ensemble may achieve the highest accuracy but be too slow or expensive for production
	• production suitability requires questions such as:
	  ○ how quickly must the model return a prediction?
	  ○ can it process real-time inputs individually?
	  ○ does it support batch processing when needed?
	  ○ how much memory and computing power does it require?
	  ○ can the system afford its operating costs at scale?
	  ○ does the accuracy improvement justify the added complexity?
	  ○ should the model be compressed, simplified, or distilled?
	  ○ can it be updated and maintained reliably?
	• model selection must balance:
	  ○ accuracy
	  ○ latency
	  ○ computational cost
	  ○ scalability
	  ○ interpretability
	  ○ maintainability
	  ○ environmental sustainability

Production Data Pipelines	**Research data is often collected and cleaned manually. This may be acceptable for a one-time experiment.**
	• production systems require automated pipelines that continuously:
	  ○ collect new data
	  ○ validate its structure and quality
	  ○ clean and transform it
	  ○ store it securely
	  ○ provide it to the model
	  ○ monitor it for unexpected changes
	  ○ support model retraining
	• important questions include:
	  ○ where does new data originate?
	  ○ how frequently does it arrive?
	  ○ how is invalid or missing data handled?
	  ○ what happens if an upstream data source changes?
	  ○ should the model be retrained daily, weekly, or conditionally?
	  ○ how will training and production data be versioned?
	  ○ how will sensitive data be protected?
	Retraining frequency should be based on model performance, data drift, cost, and business requirements—not an arbitrary schedule alone.

Systems Design and Engineering	**Research data is often collected and cleaned manually. This may be acceptable for a one-time experiment.**
	• production systems require automated pipelines that continuously:
	  ○ collect new data
	  ○ validate its structure and quality

	○ clean and transform it ○ store it securely ○ provide it to the model ○ monitor it for unexpected changes ○ support model retraining • important questions include: ○ where does new data originate? ○ how frequently does it arrive? ○ how is invalid or missing data handled? ○ what happens if an upstream data source changes? ○ should the model be retrained daily, weekly, or conditionally? ○ how will training and production data be versioned? ○ how will sensitive data be protected? **Retraining frequency should be based on model performance, data drift, cost, and business requirements—not an arbitrary schedule alone.**
Logging and Monitoring	**Logs have little value unless teams know what to collect and how to use them.** • production monitoring may cover: ○ service availability ○ response latency ○ error rates ○ resource consumption ○ input-data quality ○ data drift ○ prediction distributions

	○ model accuracy ○ output quality ○ security events ○ user engagement ○ business outcomes **Monitoring should connect technical metrics with business indicators.** • a system can remain technically available while producing increasingly poor results • alerts should be tied to defined thresholds and response procedures
Teams and Operational Processes	**A research prototype may be built by one data scientist or a small research team.** • a production ML product may require collaboration among: ○ data scientists ○ data engineers ○ software engineers ○ MLOps or DevOps engineers ○ IT operations staff ○ security and privacy specialists ○ product managers ○ legal and compliance teams ○ business stakeholders • the team needs formal procedures for: ○ storing code in a version-controlled repository ○ reviewing changes ○ running automated tests

	○ evaluating candidate models ○ approving deployments ○ releasing updates ○ monitoring production behavior ○ responding to incidents ○ rolling back failed releases **A rollback plan should exist before a new model is deployed.**
Business and Compliance Requirements	**Production failures can cause financial loss, reputational damage, regulatory violations, or user harm.** • teams should define measurable service-level objectives, such as: ○ at least 99.5% service availability ○ responses within 200 milliseconds at the 95th percentile ○ error rates below a specified threshold ○ model-quality metrics above an acceptable minimum • teams must also identify relevant requirements involving: ○ privacy ○ security ○ copyright ○ data retention ○ fairness and discrimination ○ accessibility ○ explainability ○ human oversight ○ industry-specific regulation **These requirements should be incorporated during system design rather than added after deployment.**

Major ML Failure Modes	**Stale Models and Data Drift.**
	• data drift occurs when the production-data distribution changes from the distribution used during training
	• a model may gradually lose accuracy because:
	  ○ user preferences change
	  ○ new categories or behaviors emerge
	  ○ language and cultural patterns evolve
	  ○ economic or social conditions change
	  ○ relationships between inputs and outcomes shift
	• for example:
	  ○ a music-generation model may not understand a new genre that was absent from its training data
	  ○ it may also become less relevant as user preferences change
	
	Mitigations.
	• monitor input and prediction distributions
	• track model quality over time
	• retrain periodically using recent data
	• trigger retraining when drift exceeds a threshold
	• test updated models before deployment
	• maintain a rollback-ready previous version
	
	Drift is expected in long-running ML systems.
	It should be planned for, rather than treated as an exceptional event.

Feedback Loops	**ML systems can influence the environment from which they later collect data.** • a recommendation system may create the following cycle:

	○ the model recommends particular content ○ that content receives more exposure ○ increased exposure produces more clicks ○ the system interprets those clicks as evidence of popularity ○ the model recommends the content even more frequently • this can suppress alternative content and reduce diversity • in a music system ○ repeatedly promoting safe, mainstream tracks may cause users to select those tracks more often • if those interactions are used for retraining ○ the model may become increasingly biased toward the same style **Mitigations.** • preserve exploration in recommendations • measure exposure separately from user preference • maintain diversity and novelty constraints • audit training data for model-generated effects • avoid treating all interaction data as independent evidence • use controlled experiments to evaluate recommendation policies • include human review or editorial input where appropriate
System-Integration Failures	**Failures frequently occur at the boundaries between system components.** • examples include: ○ a data pipeline stops running ○ an upstream service changes its data format ○ a required field is removed or renamed

	○ authentication credentials expire ○ a storage service becomes unavailable ○ network delays cause timeouts ○ a model receives malformed or incomplete data **Mitigations.** • validate schemas and inputs • use explicit interface contracts • test integrations automatically • monitor pipeline freshness • detect missing or malformed data • add retries, timeouts, and fallback behavior • maintain incident-response procedures
Security Breaches and Abuse	**Prompt injection.** • attackers may deliberately provide inputs designed to: ○ crash or delay the model ○ exhaust computational resources ○ reveal sensitive information ○ bypass safety controls ○ produce harmful content ○ generate copyrighted material ○ create legal or reputational problems **Mitigations.** • authenticate and authorize users • validate and sanitize inputs • apply rate limits and resource limits

	• monitor suspicious activity • test the system against adversarial inputs • filter or review high-risk outputs • protect training data and model artifacts • maintain security-incident procedures **Security must cover the complete pipeline, not only the model.**

Silent Degradation	**Silent degradation occurs when system quality declines without producing clear errors.**
	• for example:
	  ○ a music model might gradually generate more repetitive tracks because of a training problem or software defect
	  ○ the service may remain available, but users may slowly disengage
	  ○ traditional uptime monitoring may not detect this problem
	
	Mitigations.
	• monitor both model quality and business outcomes, including:
	  ○ output diversity
	  ○ repetition rates
	  ○ user engagement
	  ○ session duration
	  ○ retention
	  ○ abandonment rates
	  ○ user complaints
	  ○ conversion or task-completion rates
	  ○ long-term changes in usage

	Business key performance indicators, or KPIs, can reveal problems that technical health metrics miss.
Production-Readiness Checklist	**Before deployment, teams should verify that:** • the model meets accuracy and latency requirements • operating costs are acceptable • data pipelines are automated and monitored • training data, code, and models are versioned • deployment and rollback procedures are tested • logs support debugging and auditing • drift and output quality are monitored • security and privacy controls are active • compliance requirements are documented • failure modes have defined mitigations • technical metrics are connected to business KPIs • ownership is assigned for incidents and maintenance
Reliability in an ML System	

Geographic Redundancy	**Redundancy means maintaining multiple instances of a service so that one failure does not stop the entire system.**
	• for example:
	  ○ an organization could deploy its ML service in two geographic regions:
	  ▪ Eastern United States
	  ▪ Western United States
	  ○ if the eastern cluster becomes unavailable:

        - traffic can be redirected automatically to the western cluster
        - eastern users may experience slightly higher latency
        - **the service remains available**
- the main objective is to eliminate infrastructure-level single points of failure

Active-active deployment.

- in an active-active configuration:
    - both environments operate simultaneously
    - both receive normal production traffic
    - the workload is distributed between them
    - one environment can absorb additional traffic if the other fails
- this arrangement provides efficient resource use and rapid failover
- however, coordinating data and system state across regions can be complex

Active-passive deployment.

- in an active-passive configuration:
    - one environment handles normal production traffic
    - a second environment remains on standby
    - traffic moves to the standby environment after a failure
- this approach may be simpler to manage, but the passive environment must be tested regularly to ensure that it can take over successfully

Automatic Retries with Backoff	**Many service failures are temporary.**
	• examples include:
	  ○ brief network interruptions
	  ○ short periods of server overload
	  ○ temporary API failures
	  ○ intermittent connection errors
	• clients can retry failed requests automatically
	  ○ the client may be a:
	  ▪ user-facing application
	  ▪ backend service
	  ▪ data pipeline
	• however, immediate and repeated retries can:
	  ○ increase system load
	  ○ worsen the original failure
	
	Backoff strategy.
	• a backoff strategy increases the delay between retry attempts
	  ○ for example:
	  ▪ the first retry occurs after 1 second
	  ▪ the second occurs after 2 seconds
	  ▪ the third occurs after 4 seconds
	  ▪ later attempts continue with increasing delays
	
	Jitter.
	• a small random delay, called **jitter**, can also be added
	  ○ prevents many clients from retrying at exactly the same time

	Retries with backoff can reduce user-visible errors, although some requests may take longer to complete.
	• retries should be:
	  ○ limited to transient failures
	  ○ restricted to a maximum number of attempts
	  ○ combined with timeouts
	  ○ designed to avoid repeating unsafe operations

Circuit Breakers	**A software circuit breaker protects a system from repeatedly calling a service that is already failing.**
	• the pattern resembles an electrical circuit breaker:
	  ○ the system monitors an operation
	  ○ failures or latency exceed a defined threshold
	  ○ the circuit breaker opens
	  ○ new calls fail immediately or use a fallback
	  ○ after a waiting period, the system tests whether the service has recovered
	  ○ normal calls resume if the test succeeds
	• circuit breakers prevent repeated failures from consuming resources or spreading through dependent services
	  ○ they are particularly useful when an ML service depends on:
	  ▪ external APIs
	  ▪ feature stores
	  ▪ databases
	  ▪ model-serving endpoints
	  ▪ remote storage systems

Graceful Degradation	**Graceful degradation means providing reduced functionality when the complete service is unavailable.**
	• the principle is:
	  ○ if the system cannot provide everything, it should provide something useful rather than nothing
	• for example:
	  ○ if a personalized recommendation model fails, the application might:
	  ▪ display generally popular items
	  ▪ use cached recommendations
	  ▪ apply a simpler rule-based system
	  ▪ return results from a smaller backup model
	  ▪ disable optional personalization while preserving core functions
	
	Fallback behavior should be designed and tested before a failure occurs.

Monitoring and Automated Recovery	**Reliable systems require continuous monitoring at several levels.**
	• infrastructure metrics:
	  ○ CPU utilization
	  ○ Memory consumption
	  ○ GPU utilization
	  ○ Disk and network activity
	  ○ Queue length
	  ○ Container or server health
	• service metrics:
	  ○ availability
	  ○ request volume

  - response time
  - error count
  - timeout rate
  - retry frequency
  - circuit-breaker state
- ML metrics:
- prediction distributions.
- confidence scores.
- model accuracy.
- input-data quality.
- data drift.
- output quality.
- business metrics
- user engagement.
- task-completion rates.
- conversion.
- retention.
- abandonment.
- user complaints.

Monitoring should be connected to alerts and recovery procedures.

- automated responses may include:
  - restarting a failed service
  - replacing an unhealthy container
  - scaling computing resources
  - redirecting traffic
  - activating a fallback model
  - rolling back a recent deployment

	Automation reduces recovery time, but recovery procedures must be tested to ensure that they do not create additional failures.

Blue/Green Deployment	**Blue/green deployment is a release strategy designed to reduce downtime and deployment risk.**
	• two production environments are maintained:
	
	Environment — **Role**
	Blue — Current trusted version
	Green — New candidate version
	
	  ○ the blue environment continues serving users while the green environment is deployed and evaluated
	
	Deployment process.
	• keep the existing blue version active
	• deploy the new model or service to green
	• verify that green starts and passes basic health checks
	• send a small percentage of real traffic to green
	• monitor technical, model, and business metrics
	• increase green traffic gradually if performance remains acceptable
	• complete the cutover by directing all traffic to green
	• retain blue temporarily for rapid rollback
	
	Real traffic can reveal problems that offline testing does not detect.
	• including:
	  ○ unexpected input patterns

- increased latency
- resource bottlenecks
- integration failures
- changes in prediction quality
- changes in user behavior

Cutover.

- a cutover transfers production traffic from one environment to another
  - the cutover should occur only after the candidate model satisfies predefined requirements, such as:
    - acceptable error rates
    - acceptable response latency
    - stable resource use
    - required predictive performance
    - no significant regression in business metrics
    - no new security or compliance concerns
    - a cutover may happen gradually or all at once, depending on the system's risk and architecture

Rollback.

- if the green environment performs poorly
  - traffic can be returned immediately to blue
- this provides an important advantage:
  - the trusted system is still running
  - the previous version does not need to be redeployed
  - recovery can occur with little or no downtime
- rollback criteria should be defined before deployment

	○ teams should not wait until a failure occurs to decide what qualifies as unacceptable performance **Blue/Green Deployment and Canary Releases.** • blue/green deployment and canary releases are related but distinct **Strategy** — **Main idea** **blue/green** — maintain separate old and new environments for safe switching and rollback **canary release** — expose a small percentage of traffic to the new version before expanding deployment • the two strategies can be combined ○ a green environment can initially receive only a small share of traffic, creating a canary-style evaluation within a blue/green architecture
End-to-End ML Perspective	**A trained model is only one part of a production ML system.** • the complete lifecycle includes: ○ defining the problem ○ formulating appropriate questions ○ identifying and collecting data ○ validating and preparing the data ○ training and evaluating models ○ integrating the selected model ○ deploying the service ○ monitoring technical and model behavior ○ collecting feedback ○ retraining, updating, or retiring the model

	• a successful notebook experiment does not demonstrate that the complete system is production-ready
	  ○ production ML also requires:
	  ▪ data-engineering practices
	  ▪ software-engineering practices
	  ▪ scalable infrastructure
	  ▪ security and privacy controls
	  ▪ reliable deployment procedures
	  ▪ appropriate user experiences
	  ▪ continuous operational management

ML Systems as Sociotechnical Products	**Machine learning systems combine technology with human and organizational factors.**
	• they involve:
	  ○ users
	  ○ developers
	  ○ operators
	  ○ business stakeholders
	  ○ domain experts
	  ○ affected communities
	  ○ policies and regulations
	  ○ technical infrastructure
	
	A model can perform well statistically while the overall product still fails.
	• for example:
	  ○ users may not understand its predictions
	  ○ operators may lack an incident-response process
	  ○ the interface may encourage inappropriate reliance

	○ training data may not represent affected populations ○ the model may conflict with legal or organizational requirements **Reliability must therefore include both technical performance and responsible interaction with people and institutions.**
MLOps and Continuous Management	**Machine Learning Operations, or MLOps, applies engineering and operational practices across the ML lifecycle.** • MLOps supports: ○ reproducible training ○ automated testing ○ controlled deployment ○ model and data versioning ○ continuous monitoring ○ drift detection ○ incident response ○ safe rollback ○ ongoing model improvement **Organizations should treat an ML product as an iterative, continuously managed service—not as a one-time deliverable.**